Hibernate API

0. org.hibernate.SessionFactory API
getCurrentSession vs openSession

public Session openSession() throws HibernateException
opens NEW session from SF,which has to be explicitely closed by programmer.

OR
public Session getCurrentSession() throws HibernateException
Opens new session , if one doesn't exist , otherwise continues with the exisitng one.
Gets automatically closed upon Tx boundary(commit/rollback) or thread over(since current session is bound to current thread --mentioned in hibernate.cfg.xml property ---current_session_context_class=thread)

1. CRUD logic (save method)
API (method) of org.hibernate.Session
public Serializable save(Object o) throws HibernateException

I/P ---transient POJO ref.(not yet persistent)
Rets : Serializable ID auto generated by hibernate , associated with the PERSISTENT entity

save() method auto persists transient POJO on the DB(upon committing tx) & returns unique serializable ID generated by hibernate frmwork(if @GeneratedValue is used for id.)

2. Hibernate session API -- for data retrieval by PK
API (method) of org.hibernate.Session
public <T> T get(Class<T> class,Serializable id) throws HibernateException
T -- type of POJO
1st arg - reference to the entity class (using .class - as static variable of the java.lang.Class)
2nd arg - entity id.

Returns --- null -- if id is not found.
returns PERSISTENT pojo ref if id is found.

Usage of Hibernate Session API's get()
int id=101;
BookPOJO b1=hibSession.get(BookPOJO.class,id);//int -----Integer(auto boxing) ----upcasting------Serializable
OR
BookPOJO b1=hibSession.get(Class.forName("pojos.BookPOJO"),id);

2. Display all books info :

using HQL -- Hibernate Query Language --- Objectified version of SQL --- where table names will be replaced by POJO class names & table col names will replaced by POJO property names.
(JPA--- Java Persistence API compliant syntax --- JPQL)
eg --- SQL -- select * from books
HQL --- "from BookPOJO"
eg JPQL -- "select b from BookPOJO b"

2.1 Create Query Object --- from Session i/f
<T> org.hibernate.query.Query<T> createQuery(String hql/jpql,Class<T> resultType)

eg : Query<Book> q=hs.createQuery(hql/jpql,Book.class);

2.2. Execute query to get List of selected PERSISTENT POJOs
API of org.hibernate.query.Query i/f
(Taken from javax.persistence.TypedQuery<T>)

List<T> getResultList()

Execute a SELECT query and return the query results as a generic List<T>.
T -- type of POJO / Result

eg : hs,tx
String jpql="select b from Book b";
try {
    List<Book> l1=hs.createQuery(jpql,Book.class).getResultList();
}

Usage ---
String hql="select b from BookPOJO b";
List<BookPOJO> l1=hibSession.createQuery(hql).getResultList();

3. Passing IN params to query. & execute it.
Objective : Display all books from specified author , with price < specified price.
API from org.hibernate.query.Query i/f

Query<R> setParameter(String name,Object value)

Bind a named query parameter using its inferred Type.
name -- query param name
value -- param value.I

String hql="select b from BookPOJO b where b.price < :sp_price and b.author = :sp_auth";

How to set IN params ?
org.hibernate.query.Query<T> API
public Query<T> setPrameter(String pName,Object val)

List<Book> l1 =
hibSession.createQuery(hql,Book.class).setParameter("sp_price",user_price).setParameter("sp_auth",us
er_auth).getResultList();

Objective --Offer discount on all old books
i/p -- date , disc amt

4. Updating POJOs --- Can be done either with select followed by update or ONLY with update queries(following is eg of 2nd option
commonly known as bulk update)
Objective : Reduce price of all books with author=specified author , published before a sepecific date
String jpql = "update BookPOJO b set b.price = b.price - :disc where b.author = :au and b.publishDate <
:dt ";
set named In params
exec it (executeUpdate) ---
int updateCount= hs.createQuery(jpql).setParameter("disc", disc).setParameter("dt",

d1).executeUpdate();

---This approach is typically NOT recommended often, since it bypasses L1 cache . Cascading is not supported. Doesn't support optismistic locking directly.
Use case -- for bulk updations to be performed on a standalone (un related) table)

5. Delete operations.
API of org.hibernate.Session
--public void delete(Object o) throws HibernateException
i/p --persistent POJO ref

---POJO is marked for removal , corresponding row from DB will be deleted after comitting tx & will be removed from entity cache(l1 cache) closing of session.

OR
5.5
One can use directly "delete HQL" & perform deletions.
eg
int deletedRows = hibSession.createQuery ("delete Subscription s where s.subscriptionDate < :today").setParameter ("today", new Date ()).executeUpdate ();

Detailed API
0. SessionFactory API
getCurrentSession vs openSession

public Session openSession() throws HibernateExc
opens new session from SF,which has to be explicitely closed by prog.

public Session getCurrentSession() throws HibernateExc
Opens new session , if one doesn't exist , otherwise continues with the exisitng one.
Gets automatically closed upon Tx boundary or thread over(since current session is bound to current thread --mentioned in hibernate.cfg.xml property ---current_session_context_class ---thread)

1. Testing core api
persist ---
public void persist(Object transientRef)

if u give some non-null id (existing or non-existing) while calling persist(ref) --gives exc
org.hibernate.PersistentObjectException: detached entity passed to persist:
why its taken as detached ? ---non null id.

2.
public Serializable save(Object ref)
save --- if u give some non-null id(existing or non-existing) while calling save(ref) --doesn't give any exc.
Ignores ur passed id & creates its own id & inserts a row.

3. saveOrUpdate
public void saveOrUpdate(Object ref)
--either inserts/updates or throws exc.
null id -- fires insert (works as save)
non-null BUT existing id -- fires update (works as update)
non-null BUT non existing id -- throws StaleStateException --to indicate that we are trying to delete or

update a row that does not exist.

3.5
merge
public Object merge(Object ref)
I/P -- either transient or detached POJO ref.
O/P --Rets PERSISTENT POJO ref.

null id -- fires insert (works as save)
non-null BUT existing id -- fires update (select , update)
non-null BUT non existing id -- no exc thrown --Ignores ur passed id & creates its own id & inserts a row.(select,insert)

4. get vs load
& LazyInitilalizationException.

5. update
Session API
public void update(Object object)
Update the persistent instance with the identifier of the given detached instance.
I/P --detached POJO containing updated state.
Same POJO becomes persistent.

Exception associated :
1. org.hibernate.TransientObjectException: The given object has a null identifier:
i.e while calling update if u give null id. (transient ----X ---persistent via update)

2. org.hibernate.StaleStateException --to indicate that we are trying to delete or update a row that does not exist.
3.
org.hibernate.NonUniqueObjectException: a different object with the same identifier value was already associated with the session
eg : CustomerDaoImpl
```
public void updateCustomer(Customer cust)
{
 //cust : cust id 101
 Customer c=session.get(Customer.class,101);//c : PERSISTENT
  session.update(cust);//throws NonUniqueObjectException

}
```

6. public Object merge(Object ref)
Can Transition from transient -->persistent & detached --->persistent.
Regarding Hibernate merge
1. The state of a transient or detached instance may also be made persistent as a new persistent instance by calling merge().
2. API of Session
Object merge(Object object)
3.
Copies the state of the given object(can be passed as transient or detached) onto the persistent object with the same identifier.
3.If there is no persistent instance currently associated with the session, it will be loaded.

4.Return the persistent instance. If the given instance is unsaved, save a copy of and return it as a newly persistent instance. The given instance does not become associated with the session.
5. will not throw NonUniqueObjectException --Even If there is already persistence instance with same id in session.

7.public void evict(Object persistentPojoRef)

It detaches specified persistent object from the L1 cache.

(Removes this instance from the session cache. Changes to the instance will not be synchronized with the database.)

8.
public void clear()
It will detach all the entities associated with the session object(L1 cache) i.e all persistent entities become detached.
But Databse Connection is not returned to connection pool n L1 cache is NOT destroyed.

(Completely clears the session. Evicts all loaded instances and cancel all pending saves, updates and deletions)

9. void close()

When close() is called on session object all
the persistent objects associated with the session object become detached(l1 cache is cleared n destroyed) and also Database Connection rets to the pool.

10. void flush()

When the object is in persistent state , whatever changes we made to the object
state will be reflected in the databse only at the end of transaction.

BUT If we want to reflect the changes before the end of transaction
(i.e before commiting the transaction)
call the flush method.
(Flushing is the process of synchronizing the underlying DB state with persistable state of session cache)

11. boolean contains(Object ref)

The method indicates whether the object is
associated with session or not.(i.e is it a part of l1 cache ?)

12.
void refresh(Object ref) -- ref --persistent or detached

This method is used to get the latest data from database and make
corresponding modifications to the persistent object state.
(Re-reads the state of the given instance from the underlying database

API of org.hibernate.query.Query<T>

1. Iterator iterate() throws HibernateException

Return the query results as an Iterator. If the query contains multiple results per row, the results are returned Object[].

Entities returned --- in lazy manner

Pagination

2. Query setMaxResults(int maxResults)

Set the maximum number of rows to retrieve. If not set, there is no limit to the number of rows retrieved.

3. Query setFirstResult(int firstResult)

Set the first row to retrieve. If not set, rows will be retrieved beginnning from row 0. (NOTE row num starts from 0)

eg --- List<CustomerPOJO> l1=sess.createQuery("select c from CustomerPOJO c").setFirstResult(30).setMaxResults(10).list();

4.How to count rows & use it in pagination techniques?

```
int pageSize = 10;
   String countQ = "Select count (f.id) from Foo f";
   Query countQuery = session.createQuery(countQ);
   Long countResults = (Long) countQuery.uniqueResult();
   int lastPageNumber = (int) ((countResults / pageSize) + 1);

   Query selectQuery = session.createQuery("From Foo");
   selectQuery.setFirstResult((lastPageNumber - 1) * pageSize);
   selectQuery.setMaxResults(pageSize);
   List<Foo> lastPage = selectQuery.list();
```

5. org.hibernate.query.Query API

<T> T getSingleResult()

Executes a SELECT query that returns a single result.

Returns: Returns a single instance(persistent) that matches the query.

Throws:

NoResultException - if there is no result
NonUniqueResultException - if more than one result
IllegalStateException - if called for a JPQL UPDATE or DELETE statement

6. How to get Scrollable Result from Query?

ScrollableResults scroll(ScrollMode scrollMode) throws HibernateException

Return the query results as ScrollableResults. The scrollability of the returned results depends upon JDBC driver support for scrollable ResultSets.

Then can use methods of ScrollableResults ---first,next,last,scroll(n) .

7. How to create Named query from Session i/f?
What is a named query ?
Its a technique to group the HQL statements in single location(typically in POJOS) and lately refer them by some name whenever need to use them. It helps largely in code cleanup because these HQL statements are no longer scattered in whole code.

Fail fast: Their syntax is checked when the session factory is created, making the application fail fast in case of an error.
Reusable: They can be accessed and used from several places which increase re-usability.

eg : In POJO class, at class level , one can declare Named Queries
@Entity
@NamedQueries
({@NamedQuery(name=DepartmentEntity.GET_DEPARTMENT_BY_ID, query="select d from DepartmentEntity d where d.id = :id")}
public class Department{....}

Usgae
Department d1 = (Department)
session.getNamedQuery(DepartmentEntity.GET_DEPARTMENT_BY_ID).setInteger("id", 1);

8. How to invoke native sql from hibernate?
Query q=hs.createSQLQuery("select * from books").addEntity(BookPOJO.class);
l1 = q.list();

9. Hibernate Criteria API
A powerful and elegent alternative to HQL
Well adapted for dynamic search functionalities where complex Hibernate queries have to be generated 'on-the-fly'.

Typical steps are -- Create a criteria for POJO, add restrictions , projections ,add order & then fire query(via list() or uniqueResult())

10. For composite primary key
Rules on prim key class
Annotation -- @Embeddable (& NOT @Entity)
Must be Serializable.
Must implement hashCode & equals as per general contract.

In Owning Entity class
Add usual annotation -- @Id.

1.1 Testing core api

persist ---
public void persist(Object transientRef)

if u give some non-null id (existing or non-existing) while calling persist(ref) --gives exc org.hibernate.PersistentObjectException: detached entity passed to persist:
why its taken as detached ? ---non null id.

2.
public Serializable save(Object ref)
save --- if u give some non-null id(existing or non-existing) while calling save(ref) --doesn't give any exception
Ignores your passed id & creates its own id & inserts a row.

3. saveOrUpdate
public void saveOrUpdate(Object ref)
--either inserts/updates or throws exc.
null id -- fires insert (works as save)
non-null BUT existing id -- fires update (works as update)
non-null BUT non existing id -- throws StaleStateException --to indicate that we are trying to delete or update a row that does not exist.

3.5
merge
public Object merge(Object ref)
I/P -- either transient or detached POJO ref.
O/P --Rets PERSISTENT POJO ref.

null id -- fires insert (works as save)
non-null BUT existing id -- fires update (select , update)
non-null BUT non existing id -- no exc thrown --Ignores ur passed id & creates its own id & inserts a row.(select,insert)

4. get vs load
& LazyInitilalizationException.

5. update
Session API
public void update(Object object)
Update the persistent instance with the identifier of the given detached instance.
I/P --detached POJO containing updated state.
Same POJO becomes persistent.

Exception associated :
1. org.hibernate.TransientObjectException: The given object has a null identifier:
i.e while calling update if u give null id. (transient ----X ---persistent via update)

2. org.hibernate.StaleStateException --to indicate that we are trying to delete or update a row that does not exist.
3.
org.hibernate.NonUniqueObjectException: a different object with the same identifier value was already associated with the session

6. public Object merge(Object ref)
Can Transition from transient -->persistent & detached --->persistent.
Regarding Hibernate merge
1. The state of a transient or detached instance may also be made persistent as a new persistent instance by calling merge().
2. API of Session
Object merge(Object object)
3.
Copies the state of the given object(can be passed as transient or detached) onto the persistent object with the same identifier.
3.If there is no persistent instance currently associated with the session, it will be loaded.
4.Return the persistent instance. If the given instance is unsaved, save a copy of and return it as a newly persistent instance. The given instance does not become associated with the session.
5. will not throw NonUniqueObjectException --Even If there is already persistence instance with same id in session.

7.public void evict(Object persistentPojoRef)

It detaches a particular persistent object
detaches or disassociates from the session level cache(L1 cache)
(Remove this instance from the session cache. Changes to the instance will not be synchronized with the database.)

8.
void clear()

When clear() is called on session object all the objects associated with the session object(L1 cache) become detached.
But Databse Connection is not returned to connection pool.
(Completely clears the session. Evicts all loaded instances and cancel all pending saves, updates and deletions)

9. void close()

When close() is called on session object all
the persistent objects associated with the session object become detached(l1 cache is cleared) and also closes the Database Connection.

10. void flush()

When the object is in persistent state , whatever changes we made to the object
state will be reflected in the databse only at the end of transaction.

BUT If we want to reflect the changes before the end of transaction
(i.e before commiting the transaction)
call the flush method.
(Flushing is the process of synchronizing the underlying DB state with persistable state of session cache)

11. boolean contains(Object ref)

The method indicates whether the object is
associated with session or not.(i.e is it a part of l1 cache ?)

12.
void refresh(Object ref) -- ref --persistent or detached

This method is used to get the latest data from database and make corresponding modifications to the persistent object state.
(Re-reads the state of the given instance from the underlying database